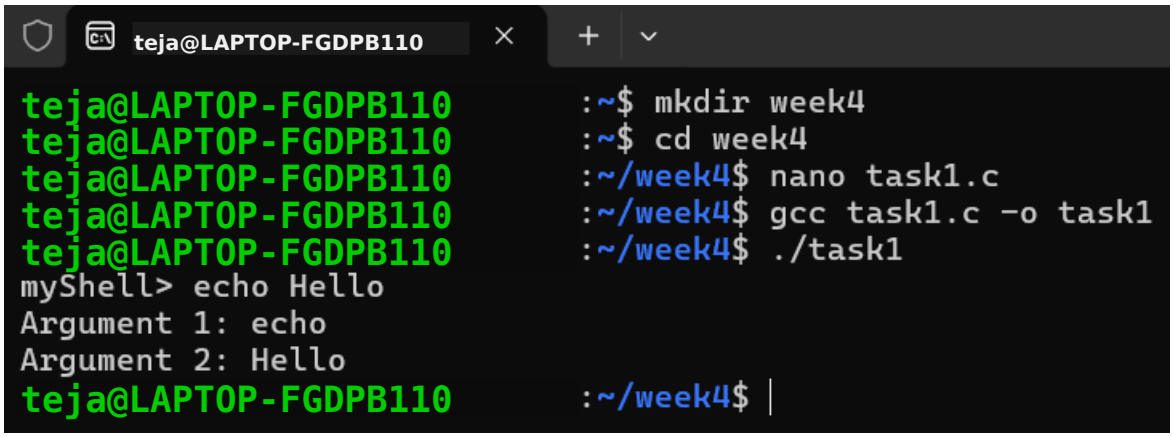
Skill Week-5

Task 1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.



```
teja@LAPTOP-FGDPB110
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];
    char *token;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    token = strtok(input, " ");

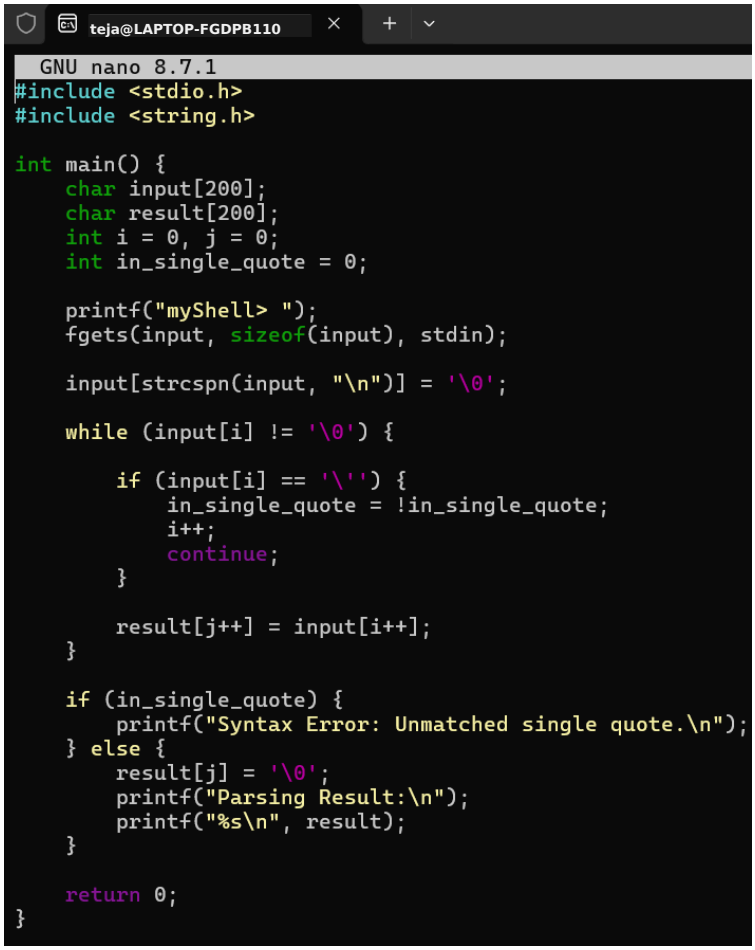
    int count = 1;

    while (token != NULL) {
        printf("Argument %d: %s\n", count, token);
        count++;
        token = strtok(NULL, " ");
    }

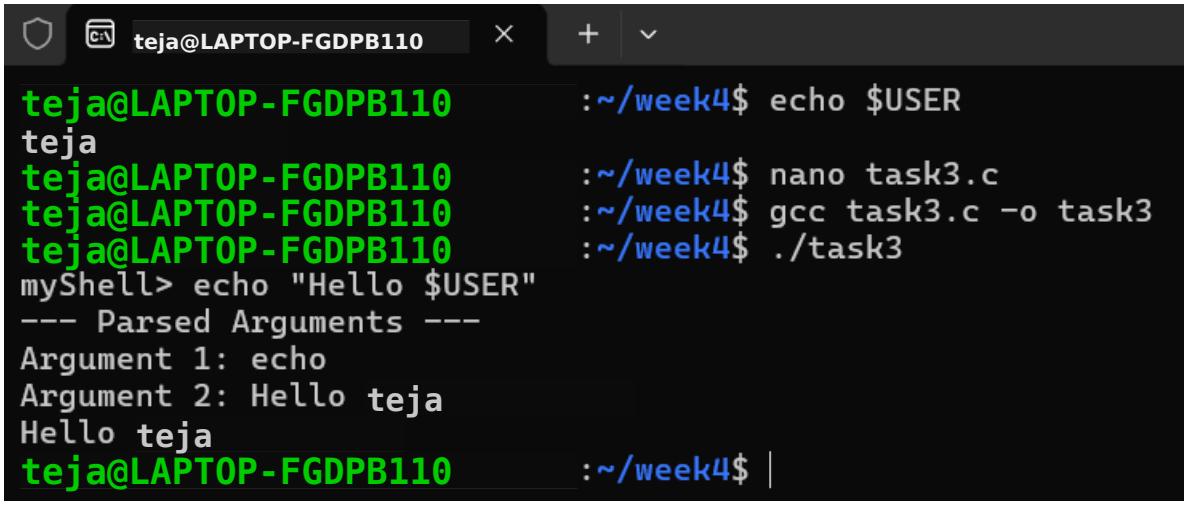
    return 0;
}
```

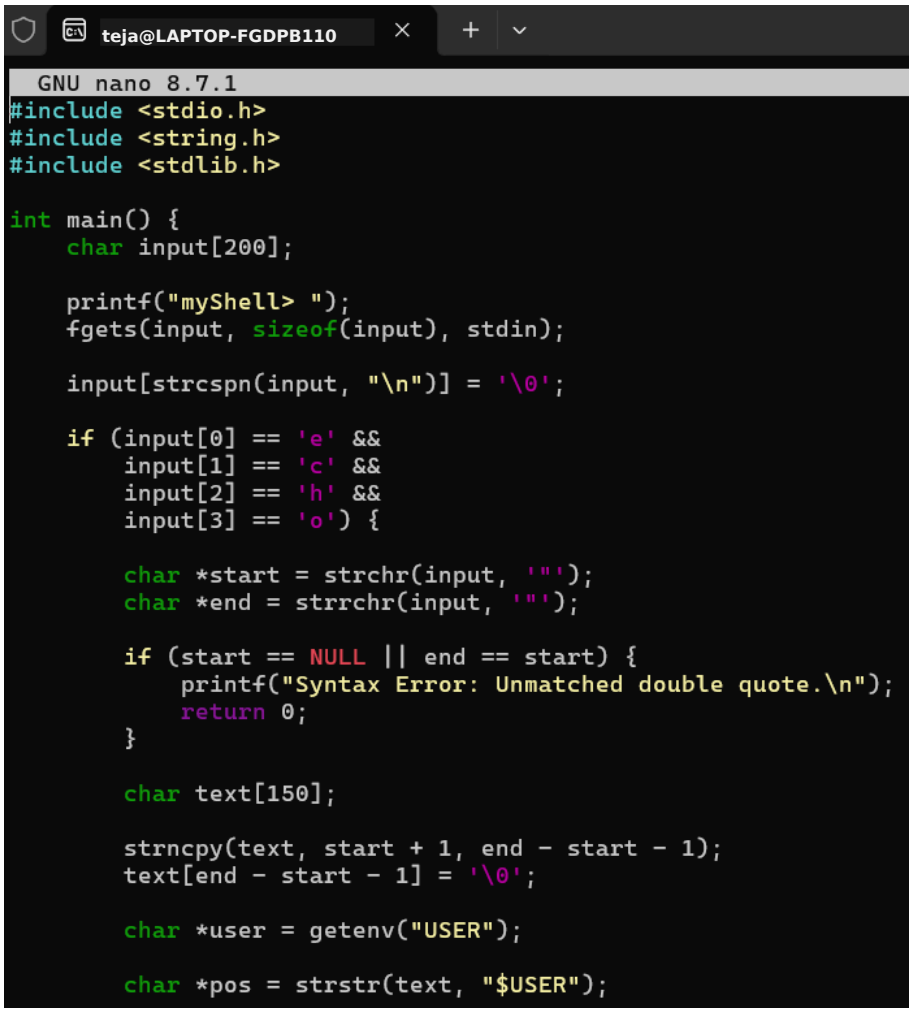
Task 2: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
teja@LAPTOP-FGDPB110
teja@LAPTOP-FGDPB110      :~/week4$ echo '$USER'
$USER
teja@LAPTOP-FGDPB110      :~/week4$ nano task2.c
teja@LAPTOP-FGDPB110      :~/week4$ gcc task2.c -o task2
teja@LAPTOP-FGDPB110      :~/week4$ ./task2
myShell> echo 'Hello World'
Parsing Result:
echo Hello World
```



Task 3:To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.


teja@LAPTOP-FGDPB110
teja@LAPTOP-FGDPB110 :~/week4$ echo $USER
teja
teja@LAPTOP-FGDPB110 :~/week4$ nano task3.c
teja@LAPTOP-FGDPB110 :~/week4$ gcc task3.c -o task3
teja@LAPTOP-FGDPB110 :~/week4$ ./task3
myShell> echo "Hello $USER"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello teja
Hello teja
teja@LAPTOP-FGDPB110 :~/week4$


teja@LAPTOP-FGDPB110
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int main() {
char input[200];
printf("myShell> ");
fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';
if (input[0] == 'e' &&
input[1] == 'c' &&
input[2] == 'h' &&
input[3] == 'o') {
char *start = strchr(input, '"');
char *end = strrchr(input, '"');
if (start == NULL || end == start) {
printf("Syntax Error: Unmatched double quote.\n");
return 0;
}
char text[150];
strncpy(text, start + 1, end - start - 1);
text[end - start - 1] = '\0';
char *user = getenv("USER");
char *pos = strstr(text, "$USER");

```c
        if (pos != NULL) {
            char result[150];

            int before = pos - text;

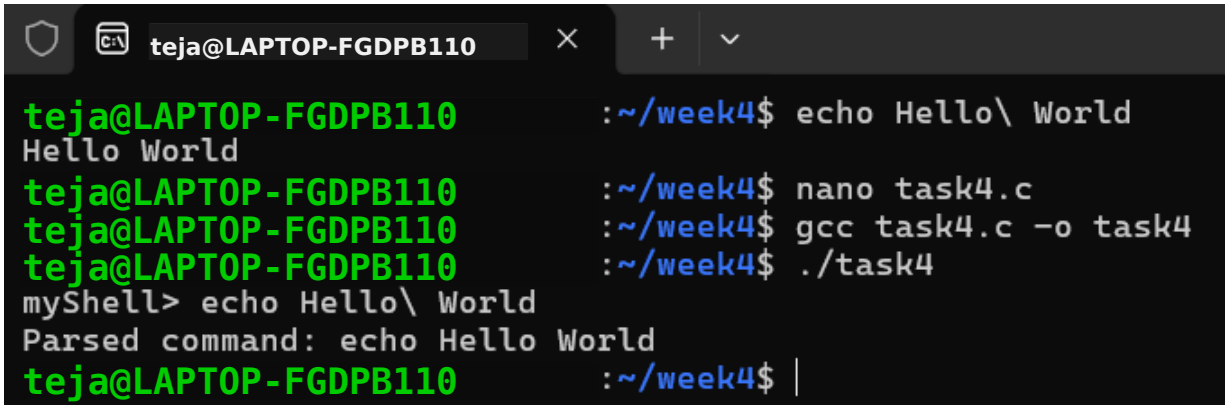
            strncpy(result, text, before);
            result[before] = '\0';

            strcat(result, user);
            strcat(result, pos + 5);

            printf("--- Parsed Arguments ---\n");
            printf("Argument 1: echo\n");
            printf("Argument 2: %s\n", result);
            printf("%s\n", result);
        } else {
            printf("Argument 1: echo\n");
            printf("Argument 2: %s\n", text);
        }
    }

    return 0;
}
```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.



teja@LAPTOP-FGDPB110

```
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char output[200];

    int i = 0;
    int j = 0;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    while (input[i] != '\0') {

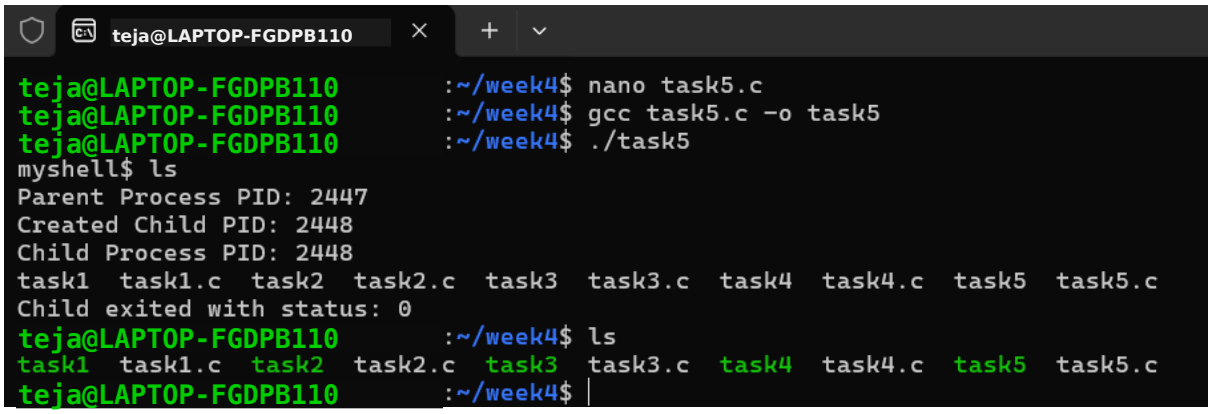
        if (input[i] == '\\' && input[i + 1] != '\0') {
            i++;
            output[j++] = input[i++];
        } else {
            output[j++] = input[i++];
        }
    }

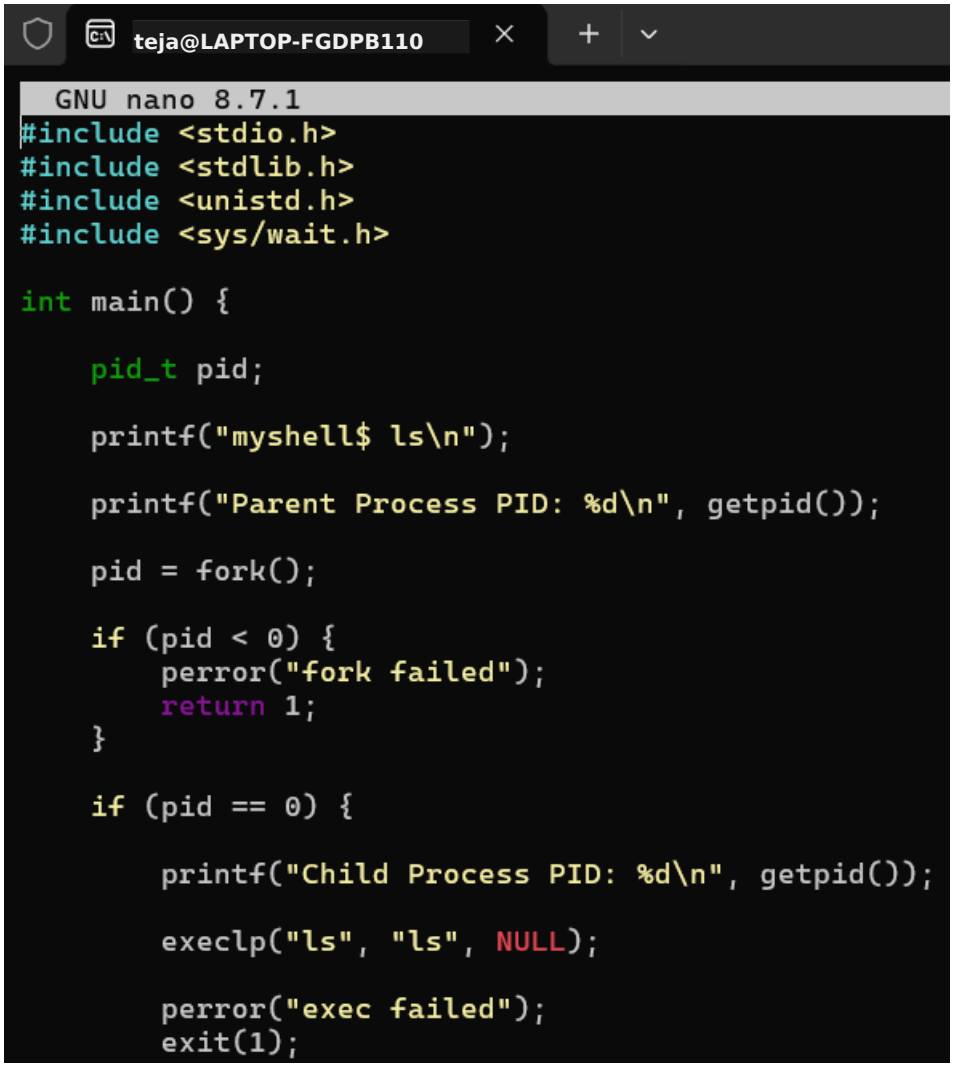
    output[j] = '\0';

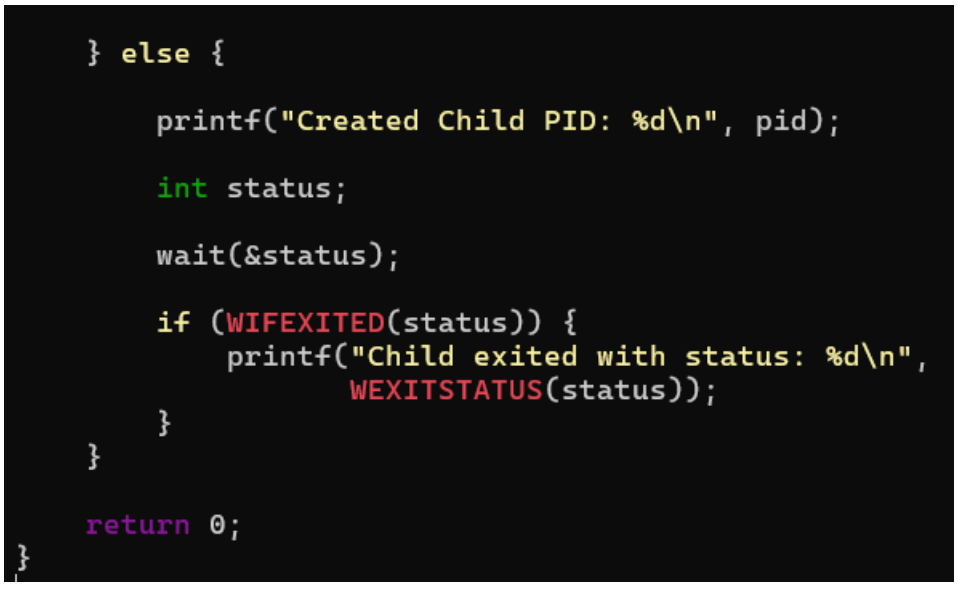
    printf("Parsed command: %s\n", output);

    return 0;
}
```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.




teja@LAPTOP-FGDPB110
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
pid_t pid;
printf("myshell$ ls\n");
printf("Parent Process PID: %d\n", getpid());
pid = fork();
if (pid < 0) {
perror("fork failed");
return 1;
}
if (pid == 0) {
printf("Child Process PID: %d\n", getpid());
execlp("ls", "ls", NULL);
perror("exec failed");
exit(1);


} else {
printf("Created Child PID: %d\n", pid);
int status;
wait(&status);
if (WIFEXITED(status)) {
printf("Child exited with status: %d\n",
WEXITSTATUS(status));
}
}
return 0;
}